

Écriture de votre première application Django, 5ème partie

Ce tutoriel commence là où le [tutoriel 4](#) s'est achevé. Nous avons construit une application Web de sondage et nous allons maintenant créer quelques tests automatisés pour cette application.



Où obtenir de l'aide :

Si vous rencontrez des problèmes dans le parcours de ce tutoriel, rendez-vous dans la section [Obtenir de l'aide de la FAQ](#).

Introduction aux tests automatisés

Que sont les tests automatisés ?

Les tests sont des routines qui vérifient le fonctionnement de votre code.

Les tests peuvent se faire à différents niveaux. Certains tests s'appliquent à un petit détail (*est-ce que tel modèle renvoie les valeurs attendues ?*), alors que d'autres examinent le fonctionnement global du logiciel (*est-ce qu'une suite d'actions d'un utilisateur sur le site produit le résultat désiré ?*). C'est le même genre de test qui a été pratiqué précédemment dans le [tutoriel 2](#), en utilisant le `shell` pour examiner le comportement d'une méthode ou en lançant l'application et en saisissant des données pour contrôler son comportement.

Ce qui est différent dans les tests *automatisés*, c'est que le travail du test est fait pour vous par le système. Vous créez une seule fois un ensemble de tests, puis au fur et à mesure des modifications de votre application, vous pouvez contrôler que votre code fonctionne toujours tel qu'il devrait, sans devoir effectuer des tests manuels fastidieux.

Pourquoi faut-il créer des tests

Ainsi donc, pourquoi créer des tests, et pourquoi maintenant ?

Vous pouvez penser que vous avez déjà assez de chats à fouetter en apprenant Python/Django et que rajouter encore une nouvelle chose à apprendre et à faire est superflu et inutile. Après tout, notre application de sondage fonctionne maintenant à satisfaction ; se préoccuper encore de créer des tests automatisés ne va pas l'améliorer. Si la création de l'application de sondage est la dernière œuvre de programmation Django que vous entreprenez, alors oui, vous pouvez vous passer d'apprendre à créer des tests automatisés. Mais si ce n'est pas le cas, alors c'est maintenant l'occasion d'apprendre cela.

Les tests vous feront gagner du temps

Jusqu'à un certain point, « contrôler que cela fonctionne apparemment » est un test satisfaisant. Dans une application plus sophistiquée, vous pouvez rencontrer des dizaines d'interactions complexes entre les différents composants.

Une modification dans n'importe lequel de ces composants pourrait avoir des conséquences inattendues sur le comportement de l'application. Contrôler que « cela semble marcher » pourrait signifier la vérification de vingt combinaisons différentes de données de test pour être sûr que vous n'avez rien cassé - vous avez certainement mieux à faire.

C'est particulièrement vrai alors que des tests automatisés pourraient faire cela pour vous en quelques secondes. Si quelque chose se passe mal, les tests vous aideront à identifier le code qui produit le comportement inapproprié.

Parfois, le fait de laisser de côté votre productif et créatif travail de programmation pour affronter la tâche ingrate et peu motivante de l'écriture de tests, peut ressembler à une corvée, spécialement lorsque vous savez que votre code fonctionne correctement.

Cependant, l'écriture de tests est bien plus rentable que de passer des heures à tester manuellement votre application ou à essayer d'identifier la cause d'un problème récemment découvert.

Les tests ne font pas qu'identifier les problèmes, ils les préviennent

C'est une erreur de penser que les tests ne sont que l'aspect négatif du développement.

Sans tests, le but ou le comportement attendu d'une application pourrait rester obscur. Même quand il s'agit de votre propre code, vous vous retrouverez parfois à fouiner un peu partout pour retrouver ce qu'il fait au juste.

Les tests changent cela ; ils éclairent votre code de l'intérieur et lorsque quelque chose va de travers, ils mettent en relief la partie qui coince, *même lorsque vous n'avez même pas réalisé que quelque chose n'allait pas*.

Les tests rendent votre code plus attractif

Vous avez peut-être créé un bout de logiciel extraordinaire, mais vous constaterez que beaucoup d'autres développeurs vont refuser de l'examiner parce qu'il ne contient pas de tests ; sans tests, ils ne lui font pas confiance. Jacob Kaplan-Moss, l'un des développeurs initiaux de Django, a dit : « Du code sans tests est par définition du code cassé ».

Le fait que d'autres développeurs veulent voir des tests dans votre logiciel avant de le prendre au sérieux est une raison supplémentaire de commencer l'écriture de tests.

Les tests aident les équipes à travailler ensemble

Les points précédents sont écrits du point de vue d'un développeur isolé maintenant une application. Les applications complexes sont maintenues par des équipes. Les tests garantissent que les collègues ne cassent votre code par mégarde (et aussi que vous ne cassiez le leur sans le vouloir). Si vous voulez gagner votre vie en tant que programmeur Django, vous devez être bon dans l'écriture de tests !

Langue : fr

Version de la documentation : 6.0

Stratégies élémentaires pour les tests ¶

Il existe de nombreuses approches pour écrire des tests.

Certains programmeurs suivent une discipline appelée « développement piloté par les tests » (test-driven development <http://fr.wikipedia.org/wiki/Test_Driven_Development>). Ils écrivent les tests avant de commencer à écrire le code. Cela peut paraître illogique, mais c'est un processus très semblable à ce que la plupart des gens font : ils décrivent un problème, puis ils écrivent du code pour le résoudre. Le développement piloté par les tests formalise le problème dans un cas de test Python.

Plus souvent, un débutant dans les tests va créer du code, puis il décidera plus tard qu'il devrait ajouter des tests. Il aurait peut-être été préférable d'écrire des tests plus tôt, mais il n'est jamais trop tard pour commencer.

Il est parfois difficile de trouver où commencer en écrivant des tests. Si vous avez écrit plusieurs milliers de lignes de code Python, choisir quoi tester n'est pas simple. Dans un tel cas, il peut être utile d'écrire le premier test au moment où vous effectuez votre prochaine modification, soit pour ajouter une nouvelle fonctionnalité ou pour corriger un bogue.

Entrons dans le vif du sujet.

Écriture du premier test ¶

Un bogue a été trouvé ¶

Heureusement, il y a un petit bogue dans l'application **polls**, tout prêt à être corrigé : la méthode **Question.was_published_recently()** renvoie **True** si l'objet **Question** a été publié durant le jour précédent (ce qui est correct), mais également si le champ **pub_date** de **Question** est dans le futur (ce qui n'est évidemment pas juste).

Confirmez l'existence du bogue en utilisant le **shell** pour contrôler la méthode sur une question dont la date se situe dans le futur :



\$ python manage.py shell

```
>>> import datetime
>>> from django.utils import timezone
>>> # create a Question instance with pub_date 30 days in the future
>>> future_question = Question(pub_date=timezone.now() + datetime.timedelta(days=30))
>>> # was it published recently?
>>> future_question.was_published_recently()
True
```

Étant donné que ce qui est dans le futur n'est pas « récent », c'est clairement une erreur.

Création d'un test pour révéler le bogue ¶

Ce que nous venons de faire dans le **shell** pour tester le problème est exactement ce que nous pouvons faire dans un test automatisé ; transformons cette opération en un test automatisé.

Un endroit conventionnel pour placer les tests d'une application est le fichier **tests.py** dans le répertoire de l'application. Le système de test va automatiquement trouver les tests dans tout fichier dont le nom commence par **test**.

Placez ce qui suit dans le fichier **tests.py** de l'application **polls**:

polls/tests.py ¶ 

import datetime

from django.test import TestCase
from django.utils import timezone

from .models import Question

class QuestionModelTests(TestCase):
 def test_was_published_recently_with_future_question(self):
 """
 was_published_recently() returns False for questions whose pub_date
 is in the future.
 """
 time = timezone.now() + datetime.timedelta(days=30)
 future_question = Question(pub_date=time)
 self.assertIs(future_question.was_published_recently(), False)

Nous venons ici de créer une sous-classe de **django.test.TestCase** contenant une méthode qui crée une instance **Question** en renseignant **pub_date** dans le futur. Nous vérifions ensuite le résultat de **was_published_recently()** qui devrait valoir **False**.

Lancement des tests ¶

Dans le terminal, nous pouvons lancer notre test :

\$ python manage.py test polls

et vous devriez voir quelque chose comme :

```
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
F
=====
FAIL: test_was_published_recently_with_future_question (polls.tests.QuestionModelTests)
-----
Traceback (most recent call last):
  File "/path/to/djangotutorial/polls/tests.py", line 16, in test_was_published_recently_with_future_question
    self.assertIs(future_question.was_published_recently(), False)
AssertionError: True is not False

-----

Ran 1 test in 0.001s

FAILED (failures=1)
Destroying test database for alias 'default'...
```



Erreur différente ?

Si vous obtenez ici une erreur **NameError**, vous avez peut-être manqué une étape à la partie 2 du tutoriel où des importations de **datetime** et de **timezone** ont été ajoutées à **polls/models.py**. Recopiez ces importations et essayez de relancer les tests.

Voici ce qui s'est passé :

- La commande **manage.py test polls** a cherché des tests dans l'application **polls** ;
- elle a trouvé une sous-classe de **django.test.TestCase** ;
- elle a créé une base de données spéciale uniquement pour les tests ;
- elle a recherché des méthodes de test, celles dont le nom commence par **test** ;
- dans **test_was_published_recently_with_future_question**, elle a créé une instance **Question** dont le champ **pub_date** est 30 jours dans le futur ;
- ... et à l'aide de la méthode **assertIs()**, elle a découvert que sa méthode **was_published_recently()** renvoyait **True**, alors que nous souhaitons qu'elle renvoie **False**.

Le test nous indique le nom du test qui a échoué ainsi que la ligne à laquelle l'échec s'est produit.

Correction du bogue ¶

Nous connaissons déjà le problème : **Question.was_published_recently()** devrait renvoyer **False** si sa **pub_date** est dans le futur. Corrigez la méthode dans **models.py** afin qu'elle ne renvoie **True** que si la date est aussi dans le passé :

polls/models.py ¶

```
def was_published_recently(self):
    now = timezone.now()
    return now - datetime.timedelta(days=1) <= self.pub_date <= now
```

puis lancez à nouveau le test :

```
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.
-----

Ran 1 test in 0.001s

OK
Destroying test database for alias 'default'...
```

Après avoir identifié un bogue, nous avons écrit un test qui le révèle et nous avons corrigé l'erreur dans le code pour que notre test réussisse.

Bien d'autres choses pourraient aller de travers avec notre application à l'avenir, mais nous pouvons être sûrs que nous n'allons pas réintroduire cette erreur par mégarde, car en lançant le test, nous serons immédiatement avertis. Nous pouvons considérer que cette petite partie de l'application est assurée de rester fonctionnelle pour toujours.

Des tests plus exhaustifs ¶

Pendant que nous y sommes, nous pouvons assurer un peu plus le fonctionnement de la méthode `was_published_recently()` ; en fait, il serait réellement embarrassant si en corrigeant un bogue, nous en avons introduit un autre.

Ajoutez deux méthodes de test supplémentaires dans la même classe pour tester le comportement de la méthode de manière plus complète :

polls/tests.py ¶



```
def test_was_published_recently_with_old_question(self):
    """
    was_published_recently() returns False for questions whose pub_date
    is older than 1 day.
    """
    time = timezone.now() - datetime.timedelta(days=1, seconds=1)
    old_question = Question(pub_date=time)
    self.assertIs(old_question.was_published_recently(), False)

def test_was_published_recently_with_recent_question(self):
    """
    was_published_recently() returns True for questions whose pub_date
    is within the last day.
    """
    time = timezone.now() - datetime.timedelta(hours=23, minutes=59, seconds=59)
    recent_question = Question(pub_date=time)
    self.assertIs(recent_question.was_published_recently(), True)
```

Et maintenant nous disposons de trois tests qui confirment que `Question.was_published_recently()` renvoie des valeurs correctes pour des questions passées, récentes et futures.

Encore une fois, `polls` est une application minimale, mais quelle que soit la complexité de son évolution ou le code avec lequel elle devra interagir, nous avons maintenant une certaine garantie que la méthode pour laquelle nous avons écrit des tests se comportera de façon cohérente.

Test d'une vue ¶

L'application de sondage est peu regardante : elle publiera toute question, y compris celles dont le champ `pub_date` est situé dans le futur. C'est à améliorer. Définir `pub_date` dans le futur devrait signifier que la question sera publiée à ce moment, mais qu'elle ne doit pas être visible avant cela.

Un test pour une vue ¶

En corrigeant le bogue ci-dessus, nous avons d'abord écrit le test, puis le code pour le corriger. En fait, c'était un exemple de développement piloté par les tests, mais l'ordre dans lequel se font les choses n'est pas fondamental.

Dans notre premier test, nous nous sommes concentrés étroitement sur le fonctionnement interne du code. Pour ce test, nous voulons contrôler son comportement tel qu'il se déroulerait avec un utilisateur depuis son navigateur.

Avant d'essayer de corriger quoi que ce soit, examinons les outils à notre disposition.

Le client de test de Django ¶

Django fournit un `Client` de test pour simuler l'interaction d'un utilisateur avec le code au niveau des vues. On peut l'utiliser dans `tests.py` ou même dans le `shell`.

Nous commencerons encore une fois par le `shell`, où nous devons faire quelques opérations qui ne seront pas nécessaires dans `tests.py`. La première est de configurer l'environnement de test dans le `shell`:



```
>>> from django.test.utils import setup_test_environment
>>> setup_test_environment()
```

`setup_test_environment()` installe un moteur de rendu de gabarit qui va nous permettre d'examiner certains attributs supplémentaires des réponses, tels que `response.context` qui n'est normalement pas disponible. Notez que cette méthode *ne crée pas* de base de données de test, ce qui signifie que ce qui suit va être appliqué à la base de données existante et que par conséquent, le résultat peut légèrement différer en fonction des questions que vous avez déjà créées. Si le réglage `TIME_ZONE` dans `settings.py` n'est pas correct, il se peut que certains résultats soient faussés. Si vous n'avez pas pensé à le régler précédemment, vérifiez-le avant de continuer.

Ensuite, il est nécessaire d'importer la classe Client de test (plus loin dans `tests.py`, nous utiliserons la classe `django.test.TestCase` qui apporte son propre client, ce qui évitera cette étape) :

```
>>> from django.test import Client
>>> # create an instance of the client for our use
>>> client = Client()
```

Ceci fait, nous pouvons demander au client de faire certaines tâches pour nous :

```
>>> # get a response from '/'
>>> response = client.get("/")
Not Found: /
>>> # we should expect a 404 from that address; if you instead see an
>>> # "Invalid HTTP_HOST header" error and a 400 response, you probably
>>> # omitted the setup_test_environment() call described earlier.
>>> response.status_code
404
>>> # on the other hand we should expect to find something at '/polls/'
>>> # we'll use 'reverse()' rather than a hardcoded URL
>>> from django.urls import reverse
>>> response = client.get(reverse("polls:index"))
>>> response.status_code
200
>>> response.content
b'\n    <ul>\n        \n        <li><a href="/polls/1/">What&#x27;s up?</a></li>\n    \n    </ul>\n\n'
>>> response.context["latest_question_list"]
<QuerySet [<Question: What's up?>]>
```

Amélioration de la vue ¶

La liste des sondages montre aussi les sondages pas encore publiés (c'est-à-dire ceux dont le champ **pub_date** est dans le futur). Corrigons cela.

Dans le tutoriel 4, nous avons introduit une vue basée sur la classe **ListView**:

polls/views.py ¶

```
class IndexView(generic.ListView):
    template_name = "polls/index.html"
    context_object_name = "latest_question_list"

    def get_queryset(self):
        """Return the last five published questions."""
        return Question.objects.order_by("-pub_date")[:5]
```

Nous devons corriger la méthode **get_queryset()** pour qu'elle vérifie aussi la date en la comparant avec **timezone.now()**. Nous devons d'abord ajouter une importation :

polls/views.py ¶

```
from django.utils import timezone
```

puis nous devons corriger la méthode **get_queryset** de cette façon :

polls/views.py ¶

```
def get_queryset(self):
    """
    Return the last five published questions (not including those set to be
    published in the future).
    """
    return Question.objects.filter(pub_date__lte=timezone.now()).order_by("-pub_date")[:5]
```

Question.objects.filter(pub_date__lte=timezone.now()) renvoie un queryset contenant les questions dont le champ **pub_date** est plus petit ou égal (c'est-à-dire plus ancien ou égal) à **timezone.now()**.

Test de la nouvelle vue ¶

Vous pouvez maintenant vérifier vous-même que tout fonctionne comme prévu en lançant **runserver** et en accédant au site depuis votre navigateur. Créez quelques instances **Question** avec des dates dans le passé et dans le futur et vérifiez que seules celles qui ont été publiées apparaissent dans la liste. Mais vous ne voulez pas faire ce travail de test manuel *chaque fois que vous effectuez une modification qui pourrait affecter ce comportement*, créons donc aussi un test basé sur le contenu de notre session **shell** précédente.

Ajoutez ce qui suit à **polls/tests.py**:

polls/tests.py ¶



```
from django.urls import reverse
```

et nous allons créer une fonction raccourci pour créer des questions, ainsi qu'une nouvelle classe de test :

polls/tests.py ¶



```
def create_question(question_text, days):
    """
    Create a question with the given `question_text` and published the
    given number of `days` offset to now (negative for questions published
    in the past, positive for questions that have yet to be published).
    """
    time = timezone.now() + datetime.timedelta(days=days)
    return Question.objects.create(question_text=question_text, pub_date=time)
```

```
class QuestionIndexViewTests(TestCase):
```

```
    def test_no_questions(self):
        """
        If no questions exist, an appropriate message is displayed.
        """
        response = self.client.get(reverse("polls:index"))
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "No polls are available.")
        self.assertQuerySetEqual(response.context["latest_question_list"], [])
```

```
    def test_past_question(self):
        """
        Questions with a pub_date in the past are displayed on the
        index page.
        """
        question = create_question(question_text="Past question.", days=-30)
        response = self.client.get(reverse("polls:index"))
        self.assertQuerySetEqual(
            response.context["latest_question_list"],
            [question],
        )
```

```
    def test_future_question(self):
        """
        Questions with a pub_date in the future aren't displayed on
        the index page.
        """
        create_question(question_text="Future question.", days=30)
        response = self.client.get(reverse("polls:index"))
        self.assertContains(response, "No polls are available.")
        self.assertQuerySetEqual(response.context["latest_question_list"], [])
```

```
    def test_future_question_and_past_question(self):
        """
        Even if both past and future questions exist, only past questions
        are displayed.
        """
        question = create_question(question_text="Past question.", days=-30)
        create_question(question_text="Future question.", days=30)
        response = self.client.get(reverse("polls:index"))
        self.assertQuerySetEqual(
            response.context["latest_question_list"],
            [question],
        )
```

```
    def test_two_past_questions(self):
        """
        The questions index page may display multiple questions.
        """
        question1 = create_question(question_text="Past question 1.", days=-30)
        question2 = create_question(question_text="Past question 2.", days=-5)
        response = self.client.get(reverse("polls:index"))
        self.assertQuerySetEqual(
            response.context["latest_question_list"],
            [question2, question1],
        )
```

Examinons plus en détails certaines de ces méthodes.

Tout d'abord, la fonction raccourci **create_question** permet d'éviter de répéter plusieurs fois le processus de création de questions.

test_no_questions ne crée aucune question, mais vérifie le message : « No polls are available. » et que la liste **latest_question_list** est vide. Notez que la classe **django.test.TestCase** fournit quelques méthodes d'assertion supplémentaires. Dans ces exemples, nous utilisons **assertContains()** et **assertQuerySetEqual()**.

Dans **test_past_question**, nous créons une question et vérifions qu'elle apparaît dans la liste.

Dans **test_future_question**, nous créons une question avec **pub_date** dans le futur. La base de données est réinitialisée pour chaque méthode de test, ce qui explique que la première question n'est plus disponible et que la page d'index n'affiche plus de question.

Et ainsi de suite. En pratique, nous utilisons les tests pour raconter des histoires d'interactions entre des saisies dans l'interface d'administration et d'un utilisateur parcourant le site, en contrôlant qu'à chaque état ou changement d'état du système, les résultats attendus apparaissent.

Test de DetailView ¶

Le code marche bien maintenant. Cependant, même si les questions futures n'apparaissent pas sur la page *index*, les utilisateurs peuvent toujours y accéder s'ils savent ou devinent la bonne URL. Nous avons donc besoin d'une contrainte semblable dans **DetailView** :

polls/views.py ¶

```
class DetailView(generic.DetailView):
    ...

    def get_queryset(self):
        """
        Excludes any questions that aren't published yet.
        """
        return Question.objects.filter(pub_date__lte=timezone.now())
```

Il est ensuite nécessaire d'ajouter quelques tests, pour contrôler qu'une question dont **pub_date** est dans le passé peut être affichée, mais que si **pub_date** est dans le futur, elle ne le sera pas :

polls/tests.py ¶

```
class QuestionDetailViewTests(TestCase):
    def test_future_question(self):
        """
        The detail view of a question with a pub_date in the future
        returns a 404 not found.
        """
        future_question = create_question(question_text="Future question.", days=5)
        url = reverse("polls:detail", args=(future_question.id,))
        response = self.client.get(url)
        self.assertEqual(response.status_code, 404)

    def test_past_question(self):
        """
        The detail view of a question with a pub_date in the past
        displays the question's text.
        """
        past_question = create_question(question_text="Past Question.", days=-5)
        url = reverse("polls:detail", args=(past_question.id,))
        response = self.client.get(url)
        self.assertContains(response, past_question.question_text)
```

Idées pour d'autres tests ¶

Nous devrions ajouter une méthode **get_queryset** similaire à **ResultsView** et créer une nouvelle classe de test pour cette vue. Elle sera très semblable à celle que nous venons de créer ; en fait, il y aura beaucoup de répétition.

Nous pourrions aussi améliorer notre application d'autres manières, en ajoutant des tests au fur et à mesure. Par exemple, il ne fait pas de sens de pouvoir publier sur le site une **Question** sans aucun choix **Choice**. Nos vues pourraient donc vérifier cela et exclure de tels objets **Question**. Les tests créeraient une question sans **Choice** et testeraient qu'elle n'est pas publiée ; de même, il s'agirait de créer une **Question** avec au moins un **Choice** et tester qu'elle est bien publiée.

Peut-être que les utilisateurs connectés dans l'interface d'administration pourraient être autorisés à voir les instances **Question** non publiées, mais pas les visiteurs ordinaires. C'est toujours le même principe, tout ce qui est ajouté au logiciel devrait être accompagné par un test, que ce soit en écrivant d'abord le test puis en écrivant le code pour réussir le test, ou en travaillant d'abord sur la logique du code et en écrivant ensuite le test pour prouver le bon fonctionnement.

À un certain stade, vous allez regarder vos tests et vous demander si votre code ne souffre pas de surabondance de tests, ce qui nous amène à :

Pour les tests, abondance de biens ne nuit pas ¶

En apparence, les tests peuvent avoir l'air de croître sans limites. À ce rythme, il y aura bientôt plus de code dans nos tests que dans notre application ; et la répétition est laide, comparée à la brièveté élégante du reste du code.

Ça n'a aucune importance. Laissez-les grandir. Dans la plupart des cas, vous pouvez écrire un test une fois et ne plus y penser. Il continuera à jouer son précieux rôle tout au long du développement de votre programme.

Les tests devront parfois être mis à jour. Supposons que nous corrigeons nos vues pour que seules les instances **Question** ayant des instances **Choice** associées puissent être publiées. Dans ce cas, de nombreux tests existants vont échouer, *ce qui nous informera exactement au sujet des tests qui devront être mis à jour* ; dans cette optique, on peut dire que les tests prennent soin d'eux-mêmes.

Au pire, au cours du développement, vous pouvez constater que certains tests deviennent redondants. Ce n'est même pas un problème. Dans les tests, la redondance est une *bonne* chose.

Tant que vos tests sont logiquement disposés, ils ne deviendront pas ingérables. Quelques bons principes à garder en tête :

- une classe de test séparée pour chaque modèle ou vue ;

- une méthode de test séparée pour chaque ensemble de conditions que vous voulez tester ;
- des noms de méthodes de test qui indiquent ce qu'elles font.

Encore plus de tests ¶

Ce tutoriel ne fait qu'introduire à quelques concepts de base des tests. Il y a encore beaucoup plus à faire et d'autres outils très utiles à votre disposition pour effectuer des choses plus perfectionnées.

Par exemple, bien que les tests présentés ici couvrent une partie de la logique interne d'un modèle et la manière dont les vues publient de l'information, vous pouvez utiliser un système basé sur de vrais navigateurs comme Selenium pour tester la façon dont le code HTML est vraiment rendu dans un navigateur. Ces outils permettent de tester plus que le comportement du seul code Django, mais aussi, par exemple, du code JavaScript. C'est assez impressionnant de voir les tests lancer un navigateur et commencer d'interagir avec votre site, comme si un être humain invisible le pilotait ! Django propose la classe `LiveServerTestCase` pour faciliter l'intégration avec des outils comme Selenium.

Si votre application est complexe, il peut être utile de lancer automatiquement les tests lors de chaque commit dans l'optique d'une intégration continue), afin que le contrôle qualité puisse être lui-même automatisé, au moins partiellement.

Une bonne façon de détecter des parties d'application non couvertes par les tests est de contrôler la couverture du code. Cela aide aussi à identifier du code fragile, ou même mort. Si vous ne pouvez pas tester un bout de code, cela signifie généralement que le code doit être réarrangé ou supprimé. Le taux de couverture aide à identifier le code inutilisé. Consultez `Intégration avec coverage.py` pour plus de détails.

Django et les tests contient des informations complètes au sujet des tests.

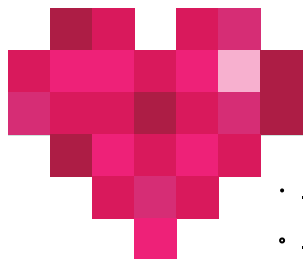
Et ensuite ? ¶

Pour des détails complets sur les tests, consultez `Django et les tests`.

Lorsque vous êtes à l'aise avec les tests de vues Django, lisez la 6ème partie de ce tutoriel pour en savoir plus sur la gestion des fichiers statiques.

Support Django!

Raphaël Titsworth-Morin donated to the Django Software Foundation to support Django development. Donate today!



Sommaire

- Écriture de votre première application Django, 5ème partie
- Introduction aux tests automatisés
- Que sont les tests automatisés ?
- Pourquoi faut-il créer des tests
 - Les tests vous feront gagner du temps
 - Les tests ne font pas qu'identifier les problèmes, ils les préviennent
 - Les tests rendent votre code plus attractif
 - Les tests aident les équipes à travailler ensemble
- Stratégies élémentaires pour les tests
- Écriture du premier test
 - Un bogue a été trouvé
 - Création d'un test pour révéler le bogue
 - Lancement des tests
 - Correction du bogue
 - Des tests plus exhaustifs
- Test d'une vue
 - Un test pour une vue
 - Le client de test de Django
 - Amélioration de la vue
 - Test de la nouvelle vue
 - Test de DetailView
 - Idées pour d'autres tests

- [Pour les tests, abondance de biens ne nuit pas](#)
- [Encore plus de tests](#)
- [Et ensuite ?](#)

Obtenir de l'aide

FAQ

Essayez la FAQ, vous y trouverez des réponses à de nombreuses questions courantes.

Index, Index des modules, or Sommaire

Pratique lorsqu'on cherche des informations précises.

Django Discord Server

Join the Django Discord Community.

Official Django Forum

Join the community on the Django Forum.

Ticket tracker

Signalez des bogues de Django ou de sa documentation dans notre système de suivi de tickets.

Télécharger :

Hors ligne (Django 6.0) : [HTML](#) | [PDF](#) | [ePub](#)

Offert par [Read the Docs](#).

Diamond and Platinum Members

Sentry

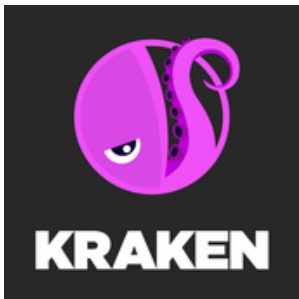
Monitor your Django Code Resolve performance bottlenecks and errors using monitoring, replays, logs and Seer an AI agent for debugging.



JetBrains

JetBrains delivers intelligent software solutions that make developers more productive by simplifying their challenging tasks, automating the routine, and helping them adopt the best development practices. PyCharm is the Python IDE for Professional Developers by JetBrains providing a complete set of tools for productive Python, Web and scientific development.





Kraken Tech

Kraken is the most-loved operating system for energy. Powered by our Utility-Grade AI™ and deep industry know-how, we help utilities transform their technology and operations so they can lead the energy transition. Delivering better outcomes from generation through distribution to supply, Kraken powers 70+ million accounts worldwide, and is on a mission to make a big, green dent in the universe.



PostHog

PostHog is an all-in-one developer platform for building successful products. We provide product analytics, web analytics, session replay, error tracking, feature flags, experiments, surveys, LLM observability, data warehouse, CDP, workflows, logs, and an AI product assistant to help debug your code, ship features faster, and keep all your usage and customer data in one stack.

Learn More

[About Django](#)

[Getting Started with Django](#)

[Team Organization](#)

[Django Software Foundation](#)

[Code of Conduct](#)

[Diversity Statement](#)

Get Involved

[Join a Group](#)

[Contribute to Django](#)

[Submit a Bug](#)

[Report a Security Issue](#)

[Individual membership](#)

Get Help

[Getting Help FAQ](#)

[Django Discord](#)

[Official Django Forum](#)

Follow Us

- GitHub
- X
- Fediverse (Mastodon)
- Bluesky
- LinkedIn
- News RSS

Support Us

- Sponsor Django
- Corporate membership
- Official merchandise store
- Benevity Workplace Giving Program